Chapter 21. Multicluster Application Deployments

Twenty chapters into this book, it should be clear that Kubernetes can be a complex topic, though of course we hope that if you have made it this far, it is less murky than it was. Given the complexities of building and running an application in a single Kubernetes cluster, why would you incur the added complexity of designing and deploying your application into multiple clusters?

The truth is that the demands of the real world mean that multicluster application deployment is a reality for most applications. There are many reasons for this, and it is likely that your application fits under at least one of these requirements.

The first requirement is one of redundancy and resiliency. Whether in the cloud or on-premise, a single datacenter is generally a single failure domain. Whether it is a hunter using a fiber-optic cable for target practice, a power outage from an ice storm, or simply a botched software rollout, any application deployed to a single location can fail completely and leave your users without recourse. In many cases, a single Kubernetes cluster is tied to a single location and thus is a single failure domain.

In some cases, especially in cloud environments, the Kubernetes cluster is designed to be *regional*. Regional clusters span across multiple independent zones and are thus resilient to the problems in the underlying infrastructure previously described. It would be tempting then to assume that such regional clusters are sufficient for resiliency and they might be except for the fact that Kubernetes itself can be a single point of failure. Any single Kubernetes cluster is tied to a specific version of Kubernetes (e.g., 1.21.3), and it is very possible for an upgrade of the cluster to break your application. From time to time Kubernetes deprecates APIs or changes the behavior of those APIs. These changes are infrequent, and the Kubernetes community takes care to make sure that they are communicated ahead of time. Additionally, despite a great deal of testing, bugs do creep into a release from time to time. Though it is unlikely for any one issue to affect your application, viewed over the lifespan of most applications (years), it's probable that your application will be affected at some point. For most applications, that's not aa\n acceptable risk.

In addition to resiliency requirements, another strong driver of multicluster deployments is some business or application need for regional affinity. For example, game servers have a strong need to be near the players to reduce network latency and improve the playing experience. Other applications may be subject to legal or regulatory requirements that demand that data be located within specific geographic regions. Since any Kubernetes cluster is tied to a specific place, these needs for application deployment to specific geographies mean that applications must span multiple clusters.

Finally, though there are numerous ways to isolate users within a single cluster (e.g., namespaces, RBAC, node pools—collections of Kubernetes nodes that are organized for different capabilities or workloads), a Kubernetes cluster is still largely a single cooperative space. For some teams and some products, the risks of a different team impacting their application, even by accident, are not worth it, and they would rather take on the complexity of managing multiple clusters.

At this point, you can see that regardless of your application, it's very likely that either now or sometime in the near future, your application will need to span multiple clusters. The rest of this chapter will help you understand how to accomplish that.

Before You Even Begin

It is critical that you have the right foundations in place in a single cluster deployment before you consider moving to multiple clusters. There is inevitably a list of to-do items that everyone has for their setup, but such shortcuts and problems are magnified in a multicluster deployment. Similarly, fixing foundational problems in your infrastructure is 10 times harder when you have 10 clusters. Furthermore, if adding an additional cluster incurs significant extra work, you will resist adding additional clusters, when (for all of the reasons already given) it is the right thing to do for your application.

When we say "foundations," what do we mean? The most important part to get right is automation. Importantly, this includes both automation to deploy your application(s), but also automation to create and manage the clusters themselves. When you have a single cluster, it is consistent with itself by definition. However, when you add clusters, you add the possibility of version skew between all of the pieces of your cluster. You could have clusters with different Kubernetes versions, different versions of your monitoring and logging agents, or even something as basic as the container runtime. All of this variance should be viewed as something that makes your life harder. Differences in your infrastructure make your system "weirder." Knowledge gained in one cluster does not transfer over to other clusters, and problems sometimes occur seemingly at random in

certain places because of this variability. One of the most important parts of maintaining a stable foundation is maintaining consistency across all of your clusters.

The only way to achieve this consistency is automation. You may think, "I always create clusters this way," but experience has taught us that this is simply not true. The next chapter discusses at length the value of infrastructure as code for managing your applications, but the same things apply to managing your clusters. Don't use a GUI or CLI tool to create your cluster. It may seem cumbersome at first to push all changes through source control and CI/CD, but the stable foundation pays significant dividends.

The same is true of the foundational components that you deploy into your clusters. These components include monitoring, logging, and security scanners, which need to be present before any application is deployed. These tools also need to be managed using infrastructure as code tools like Helm and deployed using automation.

Moving beyond the shape of your clusters, there are other aspects of consistency that are necessary. The first is using a single identity system for all of your clusters. Though Kubernetes supports simple certificate-based authentication, we strongly suggest using integrations with a global identity provider, such as Azure Active Directory or any other OpenID Connect–compatible identity provider. Ensuring that everyone uses the same identity when accessing all of the clusters is a critical part of maintaining security best practices and avoiding dangerous behaviors like sharing certificates. Additionally, most of these identity providers make available additional security controls like two-factor authentication, which enhance the security of your clusters.

Just like identity, it is also critical to ensure consistent access control to your clusters. In most clouds, this means using a cloud-based RBAC, where the RBAC roles and bindings are stored in a central cloud location rather than in the clusters themselves. Defining RBAC in a single location prevents mistakes like leaving permissions behind in one of your clusters or failing to add permissions to some single cluster. Unfortunately, if you are defining RBAC for on-premise clusters, the situation is somewhat more complicated than it is for identity. There are some solutions (e.g., Azure Arc for Kubernetes) that can provide RBAC for on-premise clusters, but if such a service is not available in your environment, defining RBAC in source control and using infrastructure as code to apply the rules to all of your clusters can ensure consistent privileges are applied across your fleet.

Similarly, when you think about defining policy for your clusters, it's critical to define those policies in a single place and have a single dashboard for viewing the compliance state of all clusters. As with RBAC, such global services are often available via your cloud provider, but for on-premise there are limited options. Using infrastructure as code for

policies as well can help close this gap and ensure that you can define your policies in a single place.

Just like setting up the right unit testing and build infrastructure is critical to your application development, setting up the right foundation for managing multiple Kubernetes clusters sets the stage for stable application deployments across a broad fleet of infrastructure. In the coming sections, we'll talk about how to build your application to operate successfully in a multicluster environment.

Starting at the Top with a Load-Balancing Approach

Once you begin to think about deploying your application into multiple locations, it becomes essential to think about how users get access to it. Typically this is through a domain name (e.g., my.company.com). Though we will spend a great deal of time discussing how to construct your application for operation in multiple locations, a more important place to start is how access is implemented. This is both because obviously enabling people to use your application is essential, but also because the design of how people access your application can improve your ability to quickly respond and reroute traffic in the case of unexpected load or failures.

Access to your application starts with a domain name. This means that the start of your multicluster load-balancing strategy starts with a DNS lookup. This DNS lookup is the first choice in your load-balancing strategy. In many traditional load-balancing approaches, this DNS lookup was used for routing traffic to specific locations. This is generally referred to as "GeoDNS." In GeoDNS, the IP address returned by the DNS lookup is tied to the physical location of the client. The IP address is generally the regional cluster that is closest to the client.

Though GeoDNS is still prevalent in many applications and may be the only possible approach for on-premise applications, it has a number of drawbacks. The first is that DNS is cached in various places throughout the internet and though you can set the time-to-live (TTL) for a DNS lookup, there are many places where this TTL is ignored in pursuit of higher performance. In a steady state operation, this caching isn't a big deal since DNS is generally pretty stable regardless of the TTL. However, it becomes a very big deal when you need to move traffic from one cluster to another; for example, in response to an outage in a particular datacenter. In such urgent cases, the fact that DNS lookups are cached can significantly extend the duration and impact of the outage. Additionally, since GeoDNS is guessing your physical location based on your client's IP address, it is

frequently confused and guesses the wrong locations when many different clients egress their traffic from the same firewall's IP address despite being in many different geographic locations.

The other alternative to using DNS to select your cluster is a load-balancing technique known as *anycast*. With anycast networking, a single static IP address is advertised from multiple locations around the internet using core routing protocols. While traditionally we think of an IP address mapping to a single machine, with anycast networking the IP address is actually a virtual IP address that is routed to a different location depending on your network location. Your traffic is routed to the "closest" location based on the distance in terms of network performance rather than geographic distance. Anycast networking generally produces better results, but it is not always available in all environments.

One final consideration as you design your load balancing is whether the load balancing happens at the TCP or HTTP level. So far we have only discussed TCP-level balancing, but for web-based applications there are significant benefits for load-balancing at the HTTP layer. If you are writing an HTTP-based application (as most applications these days are), then using a global HTTP-aware load balancer enables you to be aware of more details of the client communication. For example, you can make load-balancing decisions based on cookies that have been set in the browser. Additionally, a load balancer that is aware of the protocol can make smarter routing decisions since it sees each HTTP request instead of just a stream of bytes across a TCP connection.

Regardless of which approach you choose, ultimately the location of your service is mapped from a global DNS endpoint to a collection of regional IP addresses representing the entry point to your service. These IP addresses are generally the IP address of a Kubernetes Service or Ingress resource that you have learned about in previous chapters of the book. Once the user traffic hits that endpoint, it will flow through your cluster based on the design of your application.

Building Applications for Multiple Clusters

Once you have load balancing sorted out, the next challenge for designing a multicluster application is thinking about state. Ideally, your application doesn't require state, or all of the state is read-only. In such circumstances, there is little that you need to do to support multiple cluster deployments. Your application can be deployed individually to each of your clusters, a load balancer added to the top, and your multicluster deployment is complete. Unfortunately, for most applications there is state that must be managed in a

consistent way across the replicas of your application. If you don't handle state correctly, your users will end up with a confusing, flawed experience.

To understand how replicated state impacts user experience, let's use a simple retail shop as an example. It's obvious to see that if you only store a customer's order in one of your multiple clusters, the customer may have the unsettling experience of being unable to see their order when their requests move to a different region, either because of load balancing, or because they physically move geographies. So it is clear that a user's state needs to be replicated across regions. It may be somewhat less clear that the approach to replication also can impact the customer experience. The challenges of replicated data and customer experience is succinctly captured by this question: "Can I read my own write?" It may seem obvious that the answer should be "Yes," but achieving this is harder than it seems. Consider for example a customer who places an order on their computer, but then immediately tries to view it on their phone. They may be coming at your application from two entirely different networks and consequently landing on two completly different clusters. A user's expectation around their ability to see an order that they just placed is an example of data *consistency*.

Consistency governs how you think about replicating data. We assume that we want our data to be consistent; that is, that we will be able to read the same data regardless of where we read it from. But the complicating factor is time: how quickly must our data be consistent? And do we get any sort of error indication when it is not consistent? There are two basic models of consistency: *strong consistency*, which guarantees that a write doesn't succeed until it has been successfully replicated, and *eventual consistency*, where a write always succeeds immediately and is only guaranteed to be successfully replicated at some later point in time. Some systems also provide the ability for the client to choose their consistency needs per request. For example, Azure Cosmos DB implements *bounded consistency*, where there are some assurances about how stale data may be in an eventually consistent system. Google Cloud Spanner enables clients to specify that they are willing to tolerate stale reads in exchange for better performance.

It might seem that everyone would choose strong consistency, as it is clearly an easier model to reason about because the data is always the same everywhere. But strong consistency comes at a price. It takes much more effort to guarantee the replication at the time of the write, and many more writes will fail when replication isn't possible. Strong consistency is more expensive and can support many fewer simultaneous transactions relative to eventual consistency. Eventual consistency is cheaper and can support a much higher write load, but it is more complicated for the application developer and may expose some edge conditions to the end user. Many storage systems support only a single concurrency model. Those that support multiple concurrency models require that it be

specified when the storage system is created. Your choice of concurrency model also has significant implications for your application's design and is difficult to change. Consequently, choosing your consistency model is an important first step before designing your application for multiple environments.

NOTE

Deploying and managing replicated stateful storage is a complicated task that requires a dedicated team with domain expertise to set up, maintain, and monitor. You should strongly consider using cloud-based storage for a replicated data store so that this burden is carried by the depth of a large team at the cloud provider rather than your own teams. In an on-premise environment, you can also offload support of storage to a company that has focused expertise on running the storage solution that you choose. Only when you are at large scale does it make sense to invest in building your own team to manage storage.

Once you have determined your storage layer, the next step is to build up your application design.

Replicated Silos: The Simplest Cross-Regional Model

The simplest way to replicate your application across multiple clusters and multiple regions is simply to copy your application into every region. Each instance of your application is an exact clone and looks exactly alike no matter which cluster it is running in. Because there is a load balancer at the top spreading customer requests, and you have implemented data replication in the places where you need state, your application doesn't need to change much to support this model. Depending on the consistency model that you choose for your data, you will need to deal with the fact that data may not be replicated quickly between regions, but, especially if you opt for strong consistency, this won't require major application refactoring.

When you design your application this way, each region is its own silo. All of the data that it needs is present within the region, and once a request enters that region, it is served entirely by the containers running in that one cluster. This has significant benefits in terms of reduced complexity, but as is always the case, this comes at the cost of efficiency.

To understand how the silo approach impacts efficiency, consider an application that is distributed to a large number of geographic regions around the world in order to deliver very low latency to their users. The reality of the world is that some geographic regions have large populations and some regions have small populations. If every silo in each

cluster of the application is exactly the same, then every silo has to be sized to meet the needs of the largest geographic region. The result of this is that most replicas of the application in regional clusters are massively overprovisioned and thus cost efficiency for the application is low. The obvious solution to this excess cost is to reduce the size of the resources used by the application in the smaller geographic regions. While it might seem easy to resize your application, it's not always feasible due to bottlenecks or other requirements (e.g., maintaining at least three replicas).

Especially when taking an existing application from single cluster to multicluster, a replicated silos design is the easiest approach to use, but it is worth understanding that it comes with costs that may be sustainable initially but eventually will require your application to be refactored.

Sharding: Regional Data

As your application scales, one of the pain points that you are likely to encounter with a regional silo approach is that globally replicating all of your data becomes increasingly expensive and also increasingly wasteful. While replicating data for reliability is a good thing, it is unlikely that all of the data for your application needs to be colocated in every cluster where you deploy your application. Most users will only access your application from a small number of geographic regions.

Additionally, as your application grows around the world you may encounter regulatory and other legal requirements around data locality. There may be external restrictions on where you can store a user's data depending on their nationality or other considerations. The combination of these requirements means that eventually you will need to think about regional data sharding. Sharding your data across regions means that not all data is present in all of the clusters where your application is present and this (obviously) impacts the design of your application.

As an example of what this looks like, imagine that our application is deployed into six regional clusters (A, B, C, D, E, F). We take the dataset for our application and break the data into three subsets or *shards* (1, 2, 3).

Our data shard deployment then might look as follows:

	A	B	C	D	E	F
1	✓	-	-	✓	-	-
2	-	✓	-	-	✓	-
3	-	-	✓	-	-	✓

Each shard is present in two regions for redundancy, but each regional cluster can only serve one-third of the data. This means that you have to add an additional routing layer to your service whenever you need to access the data. The routing layer is responsible for determining whether the request needs to go to a local or cross-regional data shard.

While it might be tempting to simply implement this data routing as part of a client library that is linked into your main application, we strongly recommend that the data routing be built as a separate microservice. Introducing a new microservice might seem to introduce complexity, but it actually introduces an abstraction that simplifies things. Instead of every service in your application worrying about data routing, you have a single service that encapsulates those concerns and all other services simply access the data service. Applications that are separated into independent microservices provide significant flexibility in multicluster environments.

Better Flexibility: Microservice Routing

When we discussed the regional silo approach to multicluster application development, we gave an example of how it might reduce the cost-efficiency of your deployed multicluster application. But there are other impacts to flexibility as well. In creating the silo, you are creating at a larger scale the same sort of monoliths that containers and Kubernetes seek to break up. Furthermore, you are forcing every microservice within an application to scale at the same time to the same number of regions.

If your application is small and contained, this may make sense, but as your services become larger, and especially when they may start being shared between multiple applications, the monolithic approach to multicluster begins to significantly impact your flexibility. If a cluster is the unit of deployment and all of your CI/CD is tied to that cluster, you will force every team to adhere to the same rollout process and schedule even if it is a bad fit.

For a concrete example of this, suppose you have one very large application that is deployed to thirty clusters, and a small new application under development. It doesn't make sense to force the small team developing a new application to immediately reach the scale of your larger application, but if you are too rigid in your application design, this can be exactly what happens.

A better approach is to treat each microservice within your application as a public-facing service in terms of its application design. It may never be expected to actually be public facing, but it should have its own global load balancer as described in the previous sections, and it should manage its own data replication service. For all intents and purposes, the different microservices should be independent of each other. When a service calls into a different service, its load is balanced in the same way that an external load would be. With this abstraction in place, each team can scale and deploy their multicluster service independently, just like they do within a single cluster.

Of course, doing this for every single microservice within an application can become a significant burden on your teams and can also increase costs via the maintenance of a load balancer for each service and also possibly cross-regional network traffic. Like everything in software design, there is a trade-off between complexity and performance, and you will need to determine for your application the right places to add the isolation of a service boundary, and where it makes sense to group services into a replicated silo. Just like microservices in the single cluster context, this design is likely to change and adapt as your application changes and grows. Expecting (and designing) with this fluidity in mind will help ensure that your application can adapt without requiring massive refactoring.

Summary

Though deploying your application to multiple clusters adds complexity, the requirements and user expectations in the real world make this complexity necessary for most applications that you build. Designing your application and your infrastructure from the ground up to support multicluster application deployments will greatly increase the reliability of your application and significantly reduce the probability of a costly refactor as your application grows. One of the most important pieces of a multicluster deployment is managing the configuration and deployment of the application to the cluster. Whether your application is regional or multicluster, the following chapter will help ensure that you can quickly and reliably deploy it.

Previous chapter

< 20. Policy and Governance for Kubernetes Clusters

Next chapter

22. Organizing Your Application >